

Homework 02: Solutions

Question 1: Training and test error under a fixed design

Q1a Solution

```
n <- 100; p <- 20; R <- 200
set.seed(43202)
X_all <- matrix(rnorm(n * p), n, p)          # generated once, then held fixed
beta  <- 0.4^sqrt(1:p)
mu    <- as.vector(X_all %*% beta)

fit_first_m <- function(m, y) {              # fitted mean: intercept + first m columns
  if (m == 0) return(rep(mean(y), length(y)))
  fitted(lm(y ~ X_all[, seq_len(m), drop = FALSE]))
}

mse_train <- mse_test <- matrix(NA, R, p + 1)
for (r in 1:R) {
  y <- mu + rnorm(n); y_star <- mu + rnorm(n)
  for (m in 0:p) {
    yhat <- fit_first_m(m, y)
    mse_train[r, m + 1] <- mean((y - yhat)^2)
    mse_test[r, m + 1] <- mean((y_star - yhat)^2)
  }
}
m_seq <- 0:p
avg_train <- colMeans(mse_train); avg_test <- colMeans(mse_test)

# numerical check: training MSE never increases within a run
nonincreasing <- apply(mse_train, 1, function(row) all(diff(row) <= 1e-10))
c(all_runs_nonincreasing = all(nonincreasing),
  argmin_avg_test = which.min(avg_test) - 1)
```

all_runs_nonincreasing
1

argmin_avg_test
6

Training MSE is nonincreasing in every one of the 200 runs: adding a column enlarges the column space of the design, so the least-squares residual sum of squares cannot go up. Average training MSE falls from 1.31 at $m = 0$ to 0.79 at $m = 20$, while average test MSE bottoms out at $m = 6$ (about 1.094) and then rises to 1.22. The left panel of the figure below shows both curves.

Q1b Solution

```
B2 <- sapply(m_seq, function(m) {
  Xm <- cbind(1, X_all[, seq_len(m), drop = FALSE])
  H <- Xm %*% solve(crossprod(Xm)) %*% t(Xm)
  sum(((diag(n) - H) %*% mu)^2)
})
E_train <- B2 / n + 1 - (m_seq + 1) / n
E_test <- B2 / n + 1 + (m_seq + 1) / n
c(max_gap_train = max(abs(E_train - avg_train)),
  max_gap_test = max(abs(E_test - avg_test)), argmin_E_test = which.min(E_test) - 1)
```

```
max_gap_train  max_gap_test  argmin_E_test
      0.01723885      0.01929096      6.00000000
```

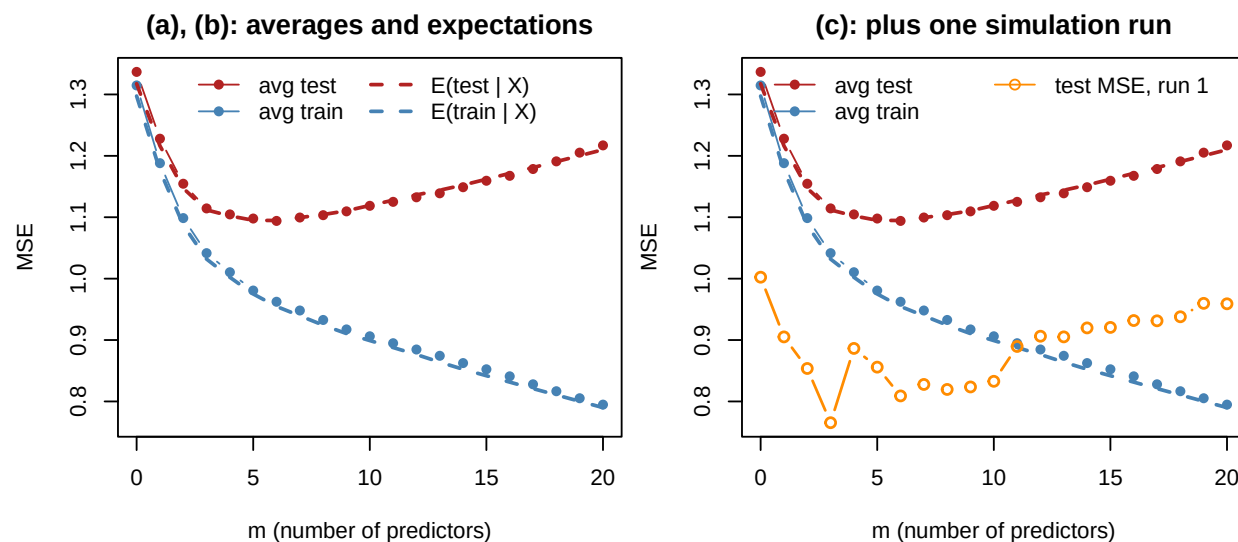


Table 1: Theory vs simulation at selected m (all 21 values were computed).

m	$B2/n$	E_{train}	avg_train	E_{test}	avg_test
0	0.307	1.297	1.315	1.317	1.337
1	0.196	1.176	1.188	1.216	1.228
2	0.117	1.087	1.099	1.147	1.155
3	0.072	1.032	1.042	1.112	1.114
4	0.052	1.002	1.010	1.102	1.105

m	B2/n	E_train	avg_train	E_test	avg_test
5	0.035	0.975	0.981	1.095	1.098
6	0.025	0.955	0.962	1.095	1.094
8	0.014	0.924	0.933	1.104	1.103
10	0.009	0.899	0.906	1.119	1.119
12	0.006	0.876	0.885	1.136	1.133
15	0.002	0.842	0.852	1.162	1.159
20	0.000	0.790	0.795	1.210	1.217

Q1c Solution

```
c(argmin_run1 = which.min(mse_test[1, ]) - 1, sd_across_runs_at_m6 = sd(mse_test[, 7]))
```

```
argmin_run1 sd_across_runs_at_m6
3.0000000    0.1530556
```

One run's test MSE carries two kinds of noise. The fitted values depend on the training noise ϵ , and every extra predictor adds a coefficient that is partly fit to that noise. The error is then measured against a single draw of ϵ^* . Both affect every point on the curve, so it jumps around its expectation with a standard deviation of roughly 0.15 at each m , about the depth of the U itself. That is why the argmin of one run is unreliable: run 1 picks $m = 3$, and its whole curve sits below even the average training curve because its test noise happened to come out small. Averaging 200 runs shrinks the noise by about $\sqrt{200} \approx 14$, which is what smooths the curve out.

For training MSE, conditional on $\mathbf{X}$,

$$E(\text{MSE}_{\text{train},m}) = \frac{B_m^2}{n} + \sigma^2 - \sigma^2 \frac{m+1}{n}, \quad E(\text{MSE}_{\text{test},m}) = \frac{B_m^2}{n} + \sigma^2 + \sigma^2 \frac{m+1}{n}.$$

The bias term B_m^2/n appears in both and can only go down as m grows. What differs is the sign on $\sigma^2(m+1)/n$. Each fitted coefficient absorbs a bit of the training noise, which lowers training MSE by σ^2/n , but that same noise gets carried into predictions on new data, where it costs σ^2/n instead. So training MSE just keeps falling and would always pick $m = p$, while test MSE trades a falling bias against a rising variance and bottoms out in the interior (at $m = 6$ here). The gap between them, $2\sigma^2(m+1)/n$, is the optimism, and training error is only useful for selection once that gap is added back.

Question 2: Prediction error at one target point

Q2a Solution

```

n <- 100; p <- 6; sigma2 <- 1
X_all <- outer(1:n, 1:p, function(i, j) sqrt(2) * cos(pi * j * (i - 0.5) / n))
beta <- rep(0.5, p); x0 <- c(0, 1, 0, 0, 1, 0)
c(max_abs_colsum = max(abs(colSums(X_all))),
  max_dev_from_I = max(abs(crossprod(X_all) / n - diag(p))))

```

```

max_abs_colsum max_dev_from_I
1.532108e-14 4.440892e-16

```

```

m_seq <- 0:p
mu0 <- sum(x0 * beta)
bias2 <- sapply(m_seq, function(m) {
  omit <- seq_len(p)[seq_len(p) > m]; sum(x0[omit] * beta[omit])^2 })
var_t <- sapply(m_seq, function(m) sigma2 / n * (1 + sum(x0[seq_len(m)]^2)))
theory <- bias2 + var_t
mu0

```

```
[1] 1
```

Why the formula has these two pieces. With this centred, orthogonal design the fitted value at $\mathbf{x}_0$ is $\hat{\mu}_{0,m} = \bar{y} + \sum_{j \leq m} x_{0j} \hat{\beta}_j$. Because the omitted columns are orthogonal to the included ones, leaving them out does not distort anything: each $\hat{\beta}_j$ in the model is still unbiased for β_j , and $\bar{y}$ is unbiased for the mean of μ , which is 0 here. So the only systematic error comes from the part of μ_0 the model has no way to represent, $\sum_{j > m} x_{0j} \beta_j$, and squaring that is the bias term. For the variance, each $\hat{\beta}_j = \mathbf{x}_j^T \mathbf{y} / n$ and $\bar{y}$ has variance σ^2 / n , and they are all uncorrelated, so the prediction variance is just σ^2 / n times $1 + \sum_{j \leq m} x_{0j}^2$.

Where the error changes. Only when a predictor with $x_{0j} \neq 0$ enters: it drops at $m = 2$ (1.01 to 0.27) and at $m = 5$ (0.27 to 0.03). It is unchanged at $m = 1, 3, 4, 6$ because those predictors have $x_{0j} = 0$.

Q2b Solution

```

set.seed(43203)
R <- 200; sq_err <- matrix(NA, R, p + 1)
predict_at_x0 <- function(m, y) {
  if (m == 0) return(mean(y))
  dat <- as.data.frame(X_all[, seq_len(m), drop = FALSE]); names(dat) <- paste0("x", seq_len(m))
  fit <- lm(y ~ ., data = cbind(y = y, dat))
  new <- as.data.frame(matrix(x0[seq_len(m)], nrow = 1)); names(new) <- names(dat)
  unname(predict(fit, newdata = new))
}
for (r in 1:R) {
  y <- as.vector(X_all %*% beta) + rnorm(n)

```

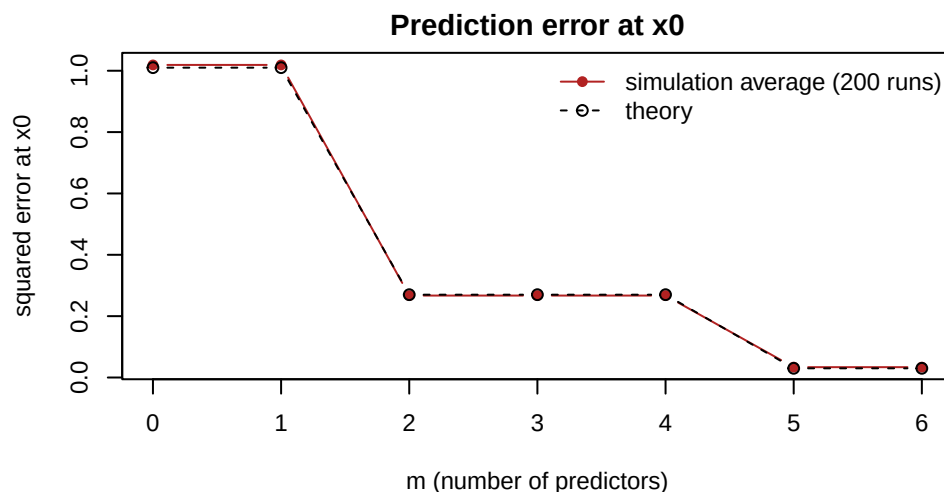
```

  for (m in m_seq) sq_err[r, m + 1] <- (predict_at_x0(m, y) - mu0)^2
}
sim_avg <- colMeans(sq_err)
m_star <- min(m_seq[theory == min(theory)]); m_star

```

[1] 5

m	squared_bias	variance	theory	simulation
0	1.00	0.01	1.01	1.0189
1	1.00	0.01	1.01	1.0189
2	0.25	0.02	0.27	0.2671
3	0.25	0.02	0.27	0.2671
4	0.25	0.02	0.27	0.2671
5	0.00	0.03	0.03	0.0338
6	0.00	0.03	0.03	0.0338



The simulation averages match the theory within Monte Carlo error at every m and reproduce the staircase: flat, drop at $m = 2$, flat, drop at $m = 5$, flat. The smallest m that minimizes the theoretical error is $m = 5$ (expected squared error 0.03). Going to $m = 6$ changes nothing because $x_{06} = 0$.

Q2c Solution

$\mu_0 = \sum_j x_{0j}\beta_j = 0 \cdot \beta_1 + 1 \cdot \beta_2 + 0 \cdot \beta_3 + 0 \cdot \beta_4 + 1 \cdot \beta_5 + 0 \cdot \beta_6$. All six coefficients are nonzero, so predictors 1, 3, 4 and 6 do affect the mean response at other covariate values, but at this $\mathbf{x}_0$ they are multiplied by zero. Omitting them causes no bias at $\mathbf{x}_0$, and with this orthogonal design including them adds no variance either (their x_{0j}^2 terms vanish).

In Question 1 the test MSE averages $(\hat{\mu}_i - \mu_i)^2$ over all n rows of $\mathbf{X}_{\text{all}}$. Every predictor with a nonzero coefficient is nonzero in many rows, so every omitted predictor adds to B_m^2/n , and every

fitted coefficient adds σ^2/n to the average variance because $\frac{1}{n} \sum_i \mathbf{x}_i^\top (\mathbf{X}_m^\top \mathbf{X}_m)^{-1} \mathbf{x}_i = \text{tr}(\mathbf{H}_m)/n = (m+1)/n$ regardless of which predictors they are. So the Q1 curves change at every m and trace a smooth U, while the target-specific error is a step function that moves only when a predictor relevant at $\mathbf{x}_0$ enters.

Question 3: The optimism correction

Q3a Solution

Let $\mathbf{M} = \mathbf{I} - \mathbf{H}$. Since $\mathbf{H}$ is symmetric and idempotent, so is $\mathbf{M}$, with $\mathbf{MH} = \mathbf{0}$ and $\text{tr}(\mathbf{M}) = n - (p+1)$.

Training error. $\mathbf{y} - \mathbf{Hy} = \mathbf{M}(\mu + \epsilon) = \mathbf{M}\mu + \mathbf{M}\epsilon$, and using $\mathbf{M}^\top \mathbf{M} = \mathbf{M}$,

$$\|\mathbf{y} - \mathbf{Hy}\|^2 = \|\mathbf{M}\mu\|^2 + 2\mu^\top \mathbf{M}\epsilon + \epsilon^\top \mathbf{M}\epsilon.$$

Conditional on $\mathbf{X}$, the first term is B^2 , the second has mean zero since $E(\epsilon \mid \mathbf{X}) = \mathbf{0}$, and the third is $\sigma^2 \text{tr}(\mathbf{M}) = \sigma^2(n - p - 1)$ by the quadratic-form identity.

Dividing by n , $E(\text{MSE}_{\text{train}} \mid \mathbf{X}) = B^2/n + \sigma^2(1 - \frac{p+1}{n})$.

Test error. $\mathbf{y}^* - \mathbf{Hy} = \mu + \epsilon^* - \mathbf{H}\mu - \mathbf{H}\epsilon = \mathbf{M}\mu + \epsilon^* - \mathbf{H}\epsilon$, so with $\mathbf{H}^\top \mathbf{H} = \mathbf{H}$,

$$\|\mathbf{y}^* - \mathbf{Hy}\|^2 = \|\mathbf{M}\mu\|^2 + \|\epsilon^*\|^2 + \epsilon^{*\top} \mathbf{H}\epsilon + 2\mu^\top \mathbf{M}\epsilon^* - 2\mu^\top \mathbf{M}\mathbf{H}\epsilon - 2\epsilon^{*\top} \mathbf{H}\epsilon.$$

The $\mathbf{MH}$ term is identically zero. The other cross terms have mean zero: ϵ^* has mean zero, and ϵ^*, ϵ are independent with mean zero so $E(\epsilon^{*\top} \mathbf{H}\epsilon) = E(\epsilon^*)^\top \mathbf{H}E(\epsilon) = 0$. For the quadratic terms, $E\|\epsilon^*\|^2 = \sigma^2 \text{tr}(\mathbf{I}_n) = n\sigma^2$ and $E(\epsilon^\top \mathbf{H}\epsilon) = \sigma^2 \text{tr}(\mathbf{H}) = \sigma^2(p+1)$. Dividing by n , $E(\text{MSE}_{\text{test}} \mid \mathbf{X}) = B^2/n + \sigma^2(1 + \frac{p+1}{n})$.

Q3b Solution

Subtracting, the bias terms cancel: $E(\text{MSE}_{\text{test}} - \text{MSE}_{\text{train}} \mid \mathbf{X}) = 2\sigma^2(p+1)/n$.

```
n <- 120; p <- 4; B2_over_n <- 0.04; sigma2 <- 1
c(E_train = B2_over_n + sigma2 * (1 - (p + 1) / n),
  E_test   = B2_over_n + sigma2 * (1 + (p + 1) / n),
  optimism = 2 * sigma2 * (p + 1) / n)
```

```
      E_train      E_test      optimism
0.99833333 1.08166667 0.08333333
```

Expected training MSE is $0.04 + 1 - 5/120 = 0.9983$, expected test MSE is $0.04 + 1 + 5/120 = 1.0817$, and the expected optimism is $10/120 = 0.0833$. The ordering holds only in expectation. For one realized pair, MSE_{test} is driven mainly by $\|\epsilon^*\|^2/n$ and $\text{MSE}_{\text{train}}$ by $\|\mathbf{M}\epsilon\|^2/n$, which are independent random quantities; if the test noise comes out small and the training noise large, the realized test MSE is the smaller one. Each MSE has standard deviation about $\sigma^2 \sqrt{2/n} \approx 0.13$, larger than the expected gap of 0.083, so reversals are common at $n = 120$.

Q3c Solution

```
RSS <- 116.4; sigma2_hat <- 0.96
MSE_test_hat <- (RSS + 2 * (p + 1) * sigma2_hat) / n
Cp <- RSS / sigma2_hat - n + 2 * (p + 1)
c(MSE_test_hat = MSE_test_hat, Cp = Cp, check = sigma2_hat * (Cp + n) / n)
```

MSE_test_hat	Cp	check
1.05	11.25	1.05

$\widehat{\text{MSE}}_{\text{test}} = (116.4 + 2 \cdot 5 \cdot 0.96)/120 = 126/120 = 1.05$ and $C_p = 116.4/0.96 - 120 + 10 = 11.25$.
 Multiplying $\widehat{\text{MSE}}_{\text{test}}$ by $n/\hat{\sigma}^2$ gives $\text{RSS}/\hat{\sigma}^2 + 2(p+1) = C_p + n$, so

$$C_p = \frac{n}{\hat{\sigma}^2} \widehat{\text{MSE}}_{\text{test}} - n, \quad \widehat{\text{MSE}}_{\text{test}} = \frac{\hat{\sigma}^2}{n} (C_p + n) \quad (\text{check: } 0.96 \cdot 131.25/120 = 1.05).$$

The correction $2(p+1)\hat{\sigma}^2/n$ is the plug-in estimate of the expected optimism from Q3b, so the corrected training MSE estimates the expected test MSE. When n and $\hat{\sigma}^2$ are common to all candidates, C_p is an increasing affine function of $\widehat{\text{MSE}}_{\text{test}}$ (slope $n/\hat{\sigma}^2$, intercept $-n$), and an increasing affine map preserves ordering, so both are minimized by the same model.

Question 4: Comparing Mallows' C_p , AIC, and BIC

Q4a Solution:

```
diabetes <- read.csv("data/diabetes.csv")
train <- diabetes[1:370, ]; test <- diabetes[371:442, ]; n <- nrow(train)
fit_A <- lm(y ~ bmi + bp + s5 + sex + s1 + s2 + s4, data = train)
fit_B <- lm(y ~ bmi + bp + s5 + sex + s1 + s2 + s4 + s6, data = train)
fit_full <- lm(y ~ age + sex + bmi + bp + s1 + s2 + s3 + s4 + s5 + s6, data = train)
rss <- function(fit) sum(resid(fit)^2)
p_A <- 7; p_B <- 8; p_full <- 10
RSS_A <- rss(fit_A); RSS_B <- rss(fit_B); RSS_full <- rss(fit_full)
df_full <- df.residual(fit_full); sigma2_hat <- RSS_full / df_full
c(RSS_A = RSS_A, RSS_B = RSS_B, RSS_full = RSS_full,
  residual_df_full = df_full, sigma2_hat = sigma2_hat)
```

RSS_A	RSS_B	RSS_full	residual_df_full
1098908.566	1089717.057	1089451.721	359.000
sigma2_hat			
3034.684			

Model A has $p = 7$, Model B $p = 8$, the full model $p = 10$. The full model has $370 - 11 = 359$ residual degrees of freedom and $\hat{\sigma}^2 = \text{RSS}_{\text{full}}/359 \approx 3034.7$.

Q4b Solution

```
criteria <- function(RSS, p) c(Cp = RSS / sigma2_hat - n + 2 * (p + 1),
                              AIC = n * log(RSS / n) + 2 * (p + 1),
                              BIC = n * log(RSS / n) + (p + 1) * log(n))
crit_tab <- rbind(`Model A` = criteria(RSS_A, p_A), `Model B` = criteria(RSS_B, p_B))
round(crit_tab, 2)
```

	Cp	AIC	BIC
Model A	8.12	2974.64	3005.95
Model B	7.09	2973.53	3008.75

```
apply(crit_tab, 2, function(v) rownames(crit_tab)[which.min(v)])
```

	Cp	AIC	BIC
"Model B"	"Model B"	"Model A"	

C_p selects Model B (7.09 vs 8.12). Reduced AIC selects Model B (2973.53 vs 2974.64). Reduced BIC selects Model A (3005.95 vs 3008.75).

Q4c Solution

```
gain_Cp <- (RSS_A - RSS_B) / sigma2_hat; gain_AIC <- n * log(RSS_A / RSS_B)
c(RSS_reduction = RSS_A - RSS_B, gain_Cp = gain_Cp, gain_AIC_BIC = gain_AIC,
  penalty_Cp_AIC = 2, penalty_BIC = log(n), t_s6 = summary(fit_B)$coefficients["s6", "t value"])
```

RSS_reduction	gain_Cp	gain_AIC_BIC	penalty_Cp_AIC	penalty_BIC
9191.508850	3.028819	3.107775	2.000000	5.913503
t_s6				
1.744979				

Adding `s6` brings RSS down by about 9192. On the C_p scale that is a gain of 3.03, and on the AIC/BIC scale it is 3.11. The catch is what each criterion charges for the extra parameter: 2 for C_p and AIC, but $\log(370) = 5.91$ for BIC. So a gain of roughly 3 is enough to convince C_p and AIC and not enough for BIC, which is really all that is going on here. The t statistic for `s6` is only about 1.7, so I would not read much into the improvement in either direction.

That does not make the winner the true model. C_p and AIC are estimating prediction error and BIC is approximating a posterior probability, so each one is only ranking the candidates it was given. The differences are about one unit, which is within the noise of these criteria, and a different 370 rows could easily flip the choice. It is also possible the real mean function has nonlinear terms or interactions that neither model includes, in which case we are just picking the less wrong of two wrong models.

Question 5: Validation and final test data

Q5a Solution

The problem is at step 2. As soon as the analyst computes test MSE for all eleven models and compares them, the test set is no longer a final check, it is a validation set. Step 3 picks m off those numbers and step 4 then reports the very number that was picked because it looked best. It is true that the test rows never went into the coefficients, but that is not the only way to leak information. Choosing m is also a decision fit to data, and here that data was the test set.

The reason the minimum comes out too optimistic is that each of the eleven test MSEs is the model's actual error plus noise, and with 72 test rows that noise is not small. When we take the smallest of eleven noisy numbers we are partly rewarding whichever model got lucky, so the reported minimum tends to sit below the real error of the model it picked. More candidates and noisier estimates make that gap wider.

```
preds <- c("age", "sex", "bmi", "bp", "s1", "s2", "s3", "s4", "s5", "s6")
form_m <- function(m) {
  if (m == 0) y ~ 1 else as.formula(paste("y ~", paste(preds[1:m], collapse = " + ")))
}
test_mse_all <- sapply(0:10, function(m)
  mean((test$y - predict(lm(form_m(m), train), test))^2))
c(analyst_choice = which.min(test_mse_all) - 1, analyst_min = min(test_mse_all))
```

```
analyst_choice    analyst_min
           9.000      2369.344
```

Q5b Solution

Procedure: (1) randomly split the 370 training rows into five folds; (2) for each m and fold k , fit the m -predictor model on the other four folds and compute the MSE on fold k ; (3) average the five fold MSEs to get $CV(m)$; (4) select $\hat{m}$ as the m with the smallest $CV(m)$; (5) refit the $\hat{m}$ -predictor model on all 370 training rows; (6) evaluate that one refitted model on rows 371 to 442, exactly once, and report that MSE as the estimate of prediction error. The test rows are touched only in step 6, after every selection decision is final.

```
set.seed(43204)
K <- 5; folds <- sample(rep(1:K, length.out = n))
cv_mse <- sapply(0:10, function(m) mean(sapply(1:K, function(k) {
  fit <- lm(form_m(m), data = train[folds != k, ])
  mean((train$y[folds == k] - predict(fit, newdata = train[folds == k, ]))^2)
})))
m_hat <- which.min(cv_mse) - 1
final_fit <- lm(form_m(m_hat), data = train) # refit on all training rows
final_test_mse <- mean((test$y - predict(final_fit, newdata = test))^2) # test data used once
c(selected_m = m_hat, final_test_MSE = final_test_mse)
```

selected_m	final_test_MSE
10.000	2484.776

Cross-validation selects $\widehat{m} = 10$, and the refitted model's MSE on the untouched test rows is 2485. The analyst's minimum over eleven test MSEs was 2369, lower than the honest figure, which is consistent with the optimistic bias from Q5a showing up in this dataset. With a different fold split or the one-standard-error rule $\widehat{m}$ could move, but whichever model is chosen, the test data still enter only once at the end.

Table 2: Test MSE of all eleven models (the analyst's misuse) and five-fold CV MSE within training.

m	analyst_test_MSE	CV_MSE
0	5822.9	5988.4
1	5527.8	5804.7
2	5527.2	5841.9
3	3719.2	3974.1
4	3246.8	3734.9
5	3234.9	3751.1
6	3207.6	3796.6
7	2636.5	3266.7
8	2624.9	3283.3
9	2369.3	3153.8
10	2484.8	3133.8

Q5c Solution

What can still be reported transparently: All eleven test MSE values can go in, with a clear note that the test set was used to compare the candidates and so effectively chose m . The minimum can be reported too, but as a number that was selected because it looked best, not as an estimate of future prediction error. Everything done on the training rows (RSS, C_p , AIC, BIC, the CV curve, and which m each of them picks) is unaffected by any of this and can be reported as is.

What cannot be reported is an unbiased estimate of the chosen model's prediction error, because there is no untouched data left to compute one from. Running CV on the training rows does not get it back: rows 371 to 442 have already been looked at for every candidate, so they can no longer act as a blind test. The only clean fix is a fresh sample from the same population, kept out of all fitting and selection and used once on the single model the training procedure produces. If that is not possible, the honest thing is to report the CV estimate and say plainly that it is an internal estimate, not an external test.